UCCS CS 3300 Intro to Software Development

SOFTWARE DEVELOPMENT REPORT

GameHub.io

04.25.2026

Table of Contents

DESCRIPTION: **1**

PROBLEM SOLVED: **1**

ENGINEERING PRINCIPLES: **1**

METHODOLOGY: 1
ARCHITECTURAL DIAGRAM: LOGICAL 2
SOFTWARE DESIGN DIAGRAM: SEQUENCE 2
PERSPECTIVE MODEL DIAGRAM: ACTIVITY 4

SOFTWARE TOOLS USED: **4**

COMPREHENSIVE TABLE: 4

ETHICS MANAGEMENT: **5**

FIREBASE 5
PLANNING PHASE 5

RISK MANAGEMENT: **5**

PROJECT RISK 5
PRODUCT RISK 6
BUSINESS RISK 6

SCHEDULE: **6**

CONFIGURATION MANAGEMENT: **8**

HOW DID WE MANAGE OUR SOFTWARE CONFIGURATION – 8
LICENSE AND OPEN-SOURCE DETAILS – 8
METRICS – 8
CODE SUMMARY – 10
LANGUAGES – 10
DIRECTORIES – 10
SYSTEM REQUIREMENTS – 10
REVIEWING REQUIREMENTS – 11
SOFTWARE VERIFICATION - 11
CURRENT CODE CONDITION - 12

TEAM MEMBERSHIP AND ROLES: **12**

GARDNER, TREY – FRONT-END DEVELOPER & BACKUP GAME DEVELOPER: 12
JAFFER, AZAM – BACK-END DEVELOPER & DATABASE ENGINEER 12
SITH, BRANDON – PAIR PROGRAMMER & UI/UX 12
ZARKOVACKI, DAVID – GAME DEVELOPER: 12

Software Development Report

Course: UCCS CS 3300 Intro to Software Engineering

Date: 04.25.2026

Project: GameHub.io

Description:

GameHub.io is a centralized adless gaming platform, which allows users to play their favorite games, track their scores, and be ranked against their friends and peers. Many current websites don't include nostalgic games from our childhood like Hangman and Snake or have features for users to easily track their progress and compare their performance against their friends. GameHub.io solves all of that by creating a centralized website with everyone's favorite nostalgic games, and friendly competition amongst friends, with an ad-free experience.

Problem Solved:

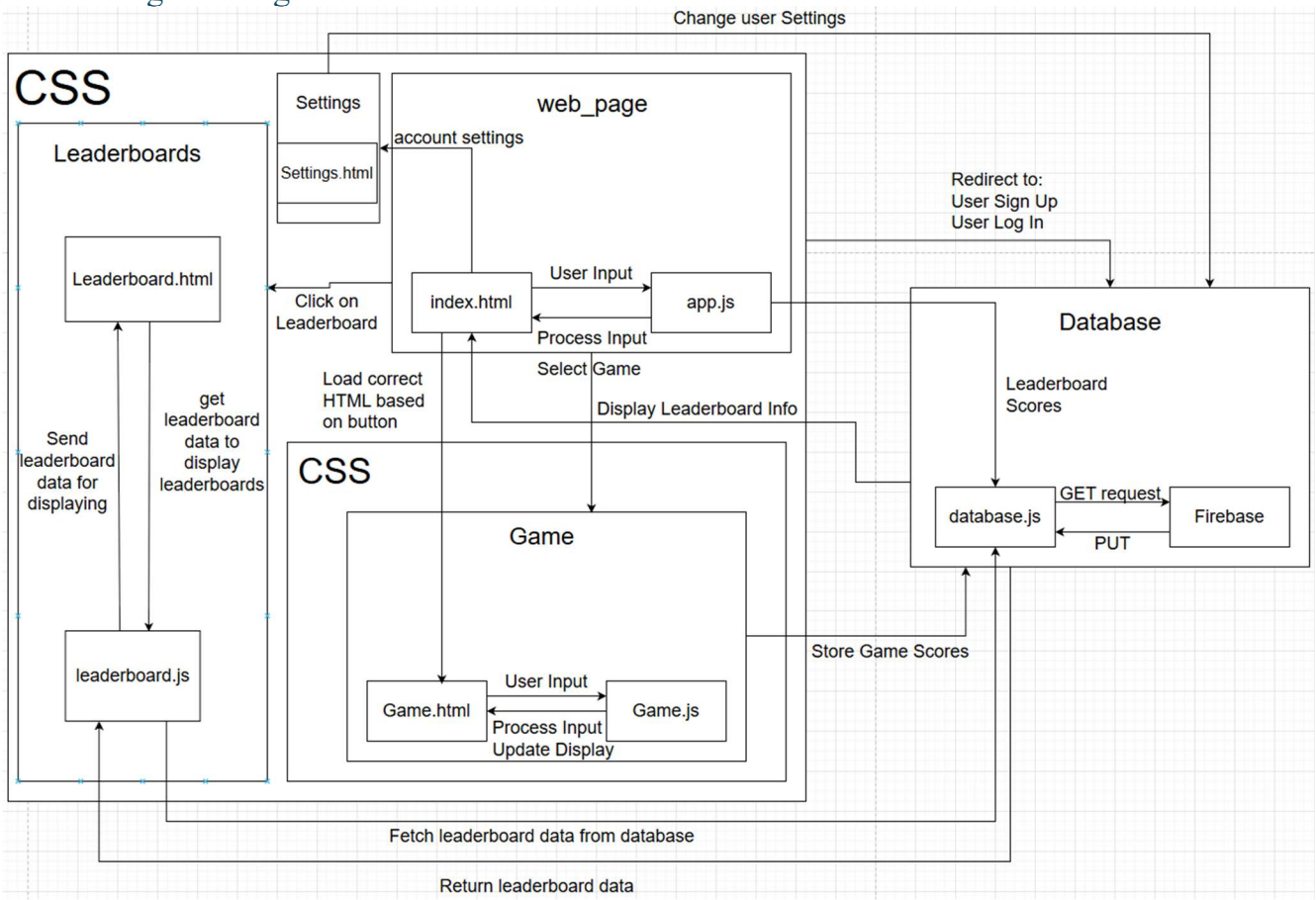
Gamehub.io provides a space for people to enjoy old school and legacy games without the annoyance of ads, emulators, or sketchy websites. We have created a space where you can play the games you love from your childhood online while competing with people around the world, just like an arcade high score. This website provides that space without collecting any personal information and without your money. We have created a free, safe, and accessible online space for gamers to chase their nostalgia.

Engineering Principles:

Methodology:

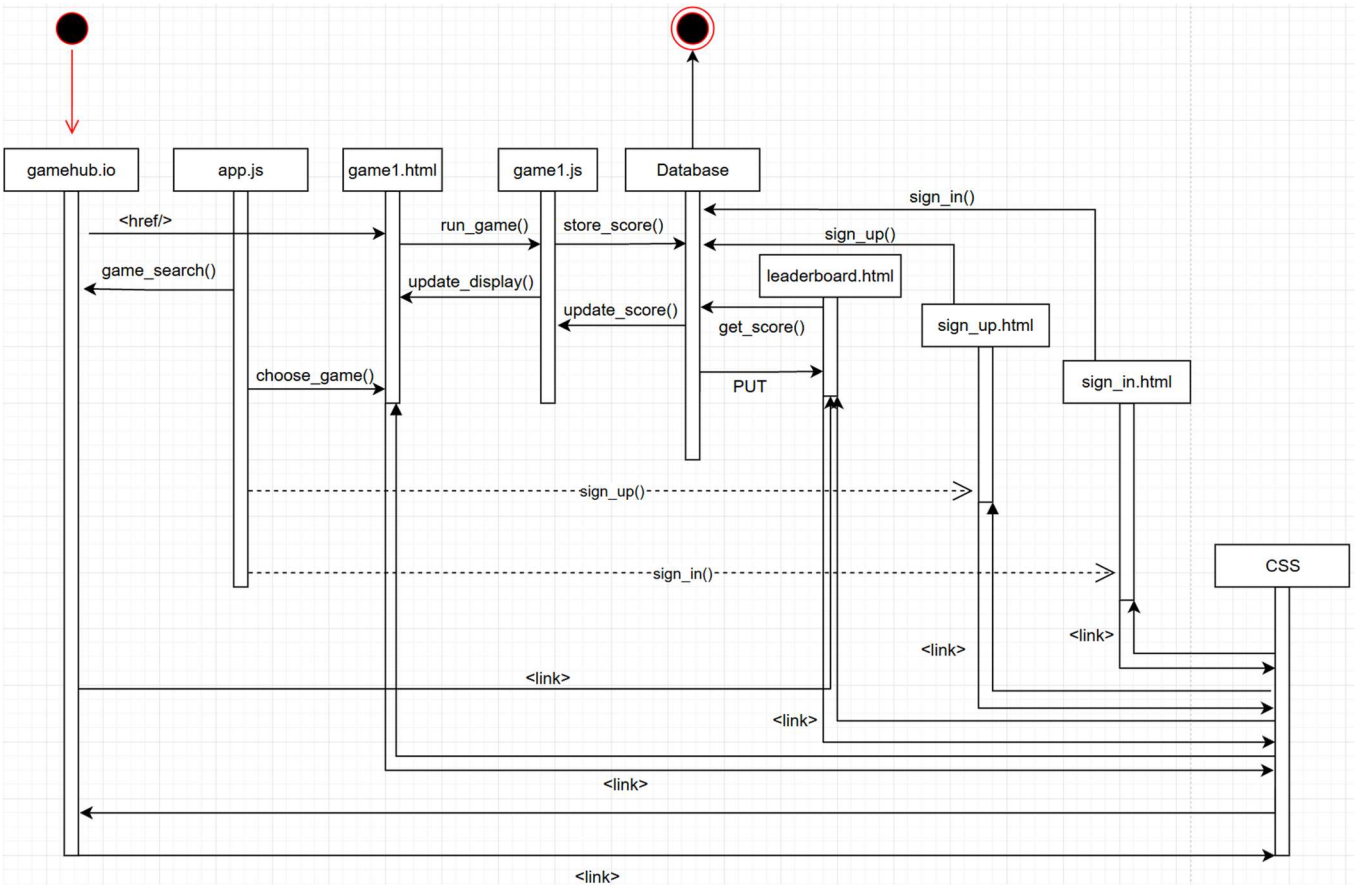
The main engineering principle we followed was the scrum framework. Since we are a smaller team, we were all responsible for developing our part and testing each other's part. This helped us stay accountable to each other, while simultaneously keeping us on schedule. We had slightly inconsistent sprint lengths depending on the requirements. For the first sprint, we started on 02/18 and ended on 02/25. During this sprint we completed the requirements that dealt with the structure of the website. The next sprint went from 02/25 to 03/01. During this sprint, we completed the basic layout of the website. We got a home page up and running with some image displays and working game links. 03/01 - 03/15 we focused on getting the user "sign in" and "sign up" pages working. We implemented a way for the web page to access the uses of data in the database. We also worked on implementing the Hangman game. From 03/15 - 04/12, we worked on implementing a different page state for signed in accounts, as well as implementing a game score and account sync for hangman. These were supposed to be in the last sprint, but we finished these requirements a few days late. We were able to get caught up, because we finished all our planned requirements for this sprint. This included creating a snake game, implementing a live leaderboard, and connecting snake score to the database. From 04/12 to 04/20, we implemented a dynamic game search, an account settings page, and a new typing test game. This pretty much completed our time working on the project; we had some overlapping non-functional requirements that we were able to mostly implement including game optimization and bug fixes when we tested each other's code. Overall, the scrum framework was a great choice for us because we incrementally produced deliverables that were added to the whole of the project.

Architectural Diagram: Logical



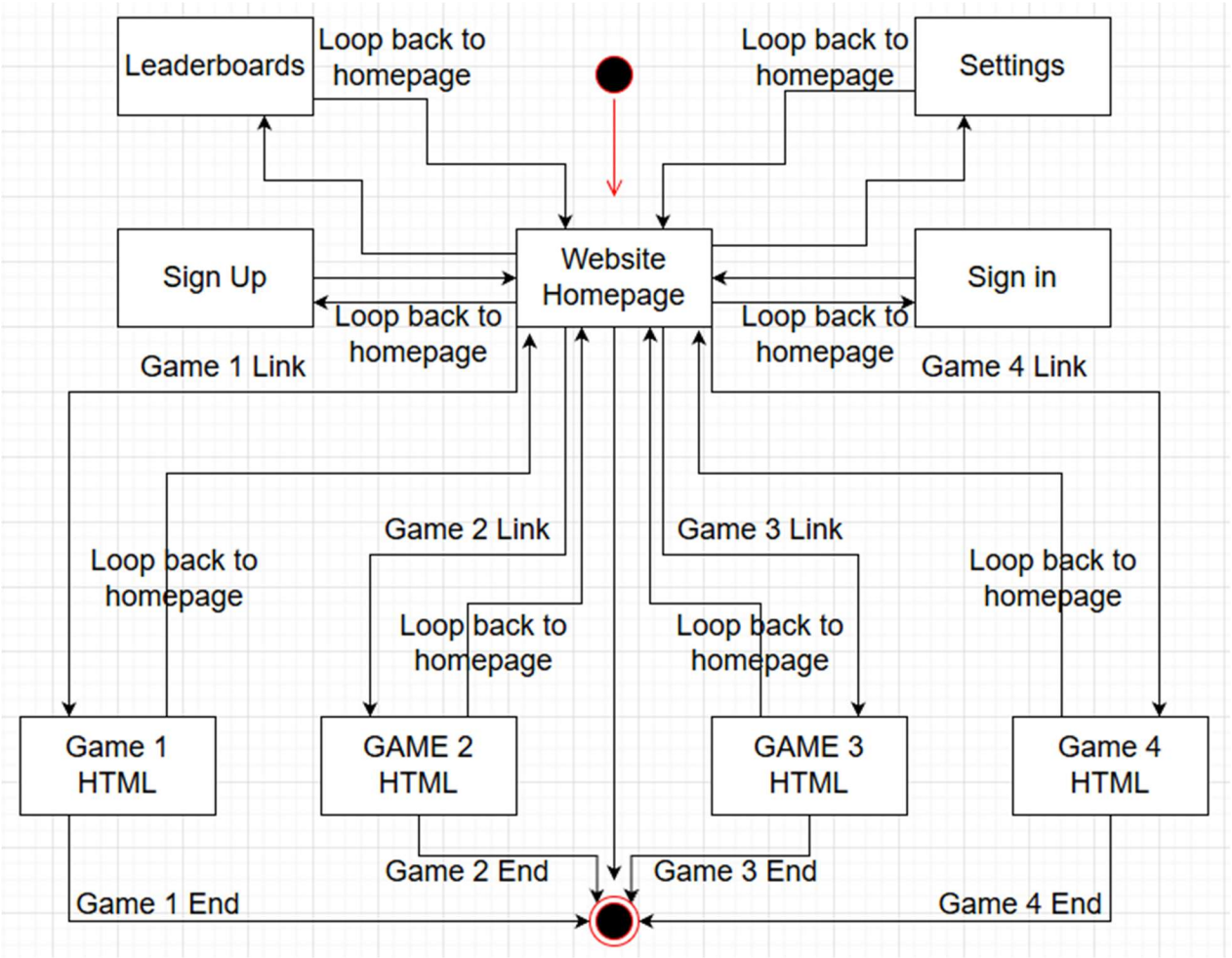
The Webpage is wrapped in a large CSS file. The homepage of the website is run by app.js. App.js does everything from processing link clicks, displaying the leaderboards, and dynamically changing based off user input (hovering over games or searching in the search bar). When you click on a game link, you will be brought to another html page. This html page is run by the game's JavaScript files. When the game ends, the game file will submit the score to the database. Each game is wrapped in its own CSS file. This CSS file is much smaller than the main one and is only used for game specific elements. From the homepage, you can visit the leaderboards page by clicking on the leaderboard preview. This brings you to the leaderboards html which is run by the leaderboards JavaScript file. This JS file requests data from the database to be displayed on the html. From the main page, if you are signed in, you can go to a settings page, and if you are not signed in, you can sign in/sign up through a sign in/sign up page. All user data including username, passwords, and game specific scores are stored in the database.

Software Design Diagram: Sequence



The Diagram shows the sequence in which our code is organized. All the HTML files are formatted by our CSS file. We start on the homepage (gamehub.io) which the user can interact with through app.js. App.js processes user input and displays all the elements of the page. From the homepage, you can select a game. Clicking the game redirects the user to the game HTML which is run by the game's JS files. When the game ends, the scores are submitted to the database. The user can then return to the homepage. From the homepage, the user can visit the leaderboard page by clicking on the leaderboard. The leaderboards HTML is run by the leaderboards JS which requests data to display from the database. If the user is not signed in, they can create an account or sign in using the sign in/sign up pages. These pages operate by requesting and putting data such as usernames and passwords into the database.

Perspective Model Diagram: Activity



This activity diagram gives an idea of how the user might navigate the webpage. The user starts at the homepage. From there, the user can sign up or sign in. Once logged in, the user can click on the leaderboards from the homepage. If they would like to edit their account, they can also visit the settings page. All these pages are looped back to the homepage. When the user is ready to play a game, they can select 1 of 4 games. Clicking on the game picture will redirect the user to different game pages. After playing the game, the user can decide to exit the website or go back to the home page. There, they can select a new game to play, view the updated leaderboards, or exit the website.

Software Tools Used:

Comprehensive Table:

Language / Tool	Version

JavaScript	ECMAScript 2025 (16th Edition)
CSS (Cascading Style Sheets)	Living Standard (No Version Iterations)
HTML5	Living Standard (No Version Iterations)
Firebase SDK	v12.12.1
SweetAlert2	v11.26.24

Ethics Management:

Firebase

From the beginning, we have been very careful in preventing any Ethical issues from arising in our website. As was pointed out to us in our original pitch presentation, there were a few concerns over the account sign up features and how privacy would be managed and protected. Much of our functionality for handling this came from Firebase, which has a basic level of security of protection for accounts made through the software. In addition, users who sign up on our website only need a username and password, keeping all of their identifying info private. While Firebase provides functions for account creation out of email and password, our project repurposes these by asking for a username instead. The only level of information shared from their account is the scores used to construct the leaderboard. We also provide an option to play games and use the website without signing in at all, which prevents any data from being stored.

Planning Phase

As software engineers, we had an ethical responsibility to keep our goals aligned with user interests and keep our website to honest standards. There were a few opportunities in the planning phase in which we had to decide on features that were ultimately turned down for the sake of these ethics. Though our entire project is based on local & live server hosting by virtue of Firebase, we have implemented heavy html structuring to model a genuine webserver off a rented domain. When first planning this concept, we considered parody ads to model the income some servers make off renting site space to companies. Columns on the left or right could showcase self-made product placement on a timer or rotating through a list. This was eventually retired on the basis of sticking to the better parts of a webserver, instead of attempting to recreate the undesirable aspects. It also goes against our jobs as software engineers to intentionally create a product that is below standards.

Risk Management:

Project Risk

When it came to Project Risks, the immediate fear came from project members within the group. Any group task has a danger of falling behind due to personal issues. Our solution was in the Project Schedule that was first compiled during our Midterm Presentation, as it was made in mind with the idea that some unavailability would be inevitable. This was our avoidance strategy, but we also had contingencies for when it wasn't enough. On a surface level, Peer Programming and our buddy system helped check over quality of work, making sure it met a standard minimum. But it also made sure that when one person had to step away, there was a second person to bat.

Another issue that was easy to run into was the matter of software. Specifically, our project was worked on across multiple types of OS and browsers. Some of us used Mac and others Windows; some firefox and some chrome. This creates an opportunity for inconsistent html code, canvas display, controls, account data, and many other aspects. Our avoidance strategy here revolved around Firebase, which provided a library of functions for implementation, along with offsite storing of account data. Both of these features removed the

possibility of any mismatches from scripts designed to work on one OS not transferring to another. The local hosting feature of Firebase also made testing these cross-platform incredibly easy, both in our buddy system and with people outside our group.

Product Risk

Product Risk came from limitations in the very two pieces of software we used. The first was Github, notorious for issues such as build fails, catastrophic merges (which I have been a part of), and more. Though this is more often than not a user error, there is still plenty of opportunity for corrupted code or loss of data with the complex ecosystem of a server. In this case, we deferred to Gitlens, our tool for helping integrate Github into VSCode. This helped us undo merges, reset to previous commits, or trash our cache to resolve conflicts. It was by far our biggest avoidance strategy, and we relied on it several times when pushes did not cleanly resolve across all our computers.

The second piece of software that saw some conflict was Firebase. Due to the way its exported functions worked, there was an inherent limitation that required us to run it from a live server, which it didn't natively provide. Instead, we worked around this limitation by the installation of another extension for VSCode named "Live Server," which allowed us to run our own local live server, providing real-time changes that were necessary for features such as our scoreboard or the account sign up/sign in.

Business Risk

Our product saw very little Business Risk, mostly because it had little opportunity to engage in a wider internet ecosystem. However, several details were noted during our time developing this project that would have created real risks for this category, had this been a more ambitious project. The biggest example of this was our name, gamehub.io. When we were far into the project, expanding past local development was considered; there was no shortage of ways to do this, and any number of services such as AWS would suffice. However, the domain name was taken and turned into a site very similar to ours, which would have forced us to adopt an entirely new name, which we would have had no contingency or plan to minimize the risk. And if we had gone this route, there were possible issues with the service we did choose. In fact, AWS itself had gone down for almost two days just last semester.

Schedule:

The Schedule, found on the next page, had a fairly bumpy ride that ultimately resulted in our minimum requirements being met. As you'll see on the next page, some items were ultimately cut off to reduce scope and meet our most important goals. We started on a very fast pace, and our sprints had gone very well as seen in our Test Reports. This was especially promising given we had already left room for any adjustments in time we might need. As April approached, we recentered our sights as we needed to make time for the deliverables at the end of the month. This loss of time is largely attributed to March, as complications in Github and Firebase forced a rollback that took time to untangle. The Godot game was axed fairly quick as

UCCS CS 3300 INTRO TO SOFTWARE ENGINEERING

TEAM TITLE:			**Current**											
SCHEDULER:	**Brandon Stih**		**Done**											
DATE:	**2/23/2026**		**Planned**											
	PLANNED Dates		**PROJECT WEEKLY SCHEDULE (actual dates)**											
ITEM	Start Date	End Date	16-Feb	23-Feb	2-Mar	9-Mar	16-Mar	23-Mar	30-Mar	6-Apr	13-Apr	20-Apr	27-Apr	4-May
PLANNING														
Establish team	1-Feb	8-Feb	4-Feb											
Initial Requirements	8-Feb	16-Feb	12-Feb											
Architecture diagram	16-Feb	2-Mar	15-Feb											
Design Webpage Layout	16-Feb	23-Feb	20-Feb											
Design Backend Structure	16-Feb	23-Feb	21-Feb											
Design Game Structure	23-Feb	2-Mar												
MID-TERM presentation	16-Feb	23-Feb												
SPRINT #1 - Path to 1.0	2-Mar	16-Mar		25-Feb			15-Mar							
IT 1 - Backend	2-Mar	16-Mar			2-Mar		15-Mar							
Data Structures	2-Mar	16-Mar		25-Feb			15-Mar							
Website Navigation	2-Mar	8-Mar			2-Mar		15-Mar							
Godot Support	2-Mar	8-Mar												
IT 2 - Frontend	2-Mar	16-Mar			2-Mar		15-Mar							
Test Game	2-Mar	16-Mar			2-Mar		15-Mar							
Display Canvas	2-Mar	8-Mar			2-Mar		15-Mar							
SPRINT #2 - 1.0 Features	16-Mar	23-Mar					15-Mar		31-Mar					
IT 1 - Backend	16-Mar	23-Mar					15-Mar				16-Mar			
Pull/Store User Data	16-Mar	23-Mar					15-Mar				16-Apr			
Account Verification	16-Mar	23-Mar					15-Mar				16-Apr			
Display Canvas Hangman	16-Mar	23-Mar					15-Mar	16-Mar						
IT 2 - Frontend	16-Mar	23-Mar					15-Mar	16-Mar						
Hangman Game Build	16-Mar	23-Mar					15-Mar	16-Mar						
Build Unit Test Plan	16-Mar	23-Mar					15-Mar	16-Mar						
Sign-up/Sign-in Function	16-Mar	23-Mar					15-Mar				16-Mar			
IT 3 - Godot Prototyping	16-Mar	23-Mar					15-Apr		N/A					
Data Structures	16-Mar	23-Mar					15-Apr		N/A					
Early Scene Building	16-Mar	23-Mar					15-Apr		N/A					
Simple Art Assets	16-Mar	23-Mar					15-Apr		N/A					
SPRINT #3 - Road to 2.0	23-Mar	6-Apr								12-Apr		20-Apr		
IT 1 - Backend	23-Mar	6-Apr								12-Apr	13-Apr			
Canvas Display (New Game	30-Mar	6-Apr								12-Apr	13-Apr			
Account Sync (New Game)	30-Mar	6-Apr								12-Apr		18-Apr		
Account Settings Page	23-Mar	30-Mar								12-Apr	13-Apr			
Dynamic Game Search	23-Mar	30-Mar								12-Apr	13-Apr			
IT2 - Frontend	23-Mar	6-Apr										20-Apr		
Snake Game Build	23-Mar	6-Apr								12-Apr		20-Apr		
Leaderboard Data	23-Mar	30-Mar								12-Apr	18-Apr			
Typing Test Game Build	23-Mar	6-Apr								12-Apr	13-Apr			
SPRINT #3 - 2.0 Features	6-Apr	20-Apr								5-Apr		19-Apr		
IT 1 - Backend	6-Apr	20-Apr								5-Apr		19-Apr		
Optimization of Prev Games	6-Apr	20-Apr								5-Apr	6-Apr			
Rework Webpage Anim.	6-Apr	20-Apr									18-Apr	19-Apr		
Implement Light/dark Theme	6-Apr	13-Apr									N/A			
User Cache for auto-log	6-Apr	13-Apr									11-Apr	15-Apr		
IT2 - Frontend	6-Apr	20-Apr									15-Apr		22-Apr	
Bomb Field Build	6-Apr	13-Apr									17-Apr		N/A	
5th Game Build	6-Apr	13-Apr									N/A			
6th Game Build	13-Apr	20-Apr									N/A			
Dynamic Leaderboard Data	13-Apr	20-Apr										22-Apr	22-Apr	
Password Reset	6-Apr	13-Apr									N/A			
Artifact/ Documentation														
Requirement Spreadsheet	23-Feb	4-May												22-Apr
Project Schedule	23-Feb	4-May												22-Apr
Software Test Report (input conclusions from every executed test)	23-Feb	4-May												
Users Manual	6-Apr	4-May												
Future Developer's Document	6-Apr	4-May												
Software Development Report	6-Apr	4-May												
Code														
Source Code	23-Feb	4-May												
License	6-Apr	4-May												
UCCS TA software read-me file	6-Apr	4-May												
FINALpresentation	27-Apr	4-May												

Configuration Management:

How did we manage our software configuration –

As a team, our primary tactic for managing our software configuration was to limit the number of dependencies, moving parts, and external tools as much as possible. Throughout the production of this software, we stuck to one version of all our languages, tools, frameworks, and dependencies. We also utilized importing dependencies through API calls live as opposed to trying to store, manage, and package dependencies for the program. To ensure the flow of production was smooth and consistent for all developers, we set up identical environments in VS Code. Each environment was setup with the same server extension, language versions, and code formatting tools to create a consistent developer environment.

License and open-source details –

Our product is licensed under an MIT License. This is possible because all the code, art, and functionality of our website and games is originally created by our team of developers. We have games on our website that have been heavily inspired by games that exist in public domain such as –

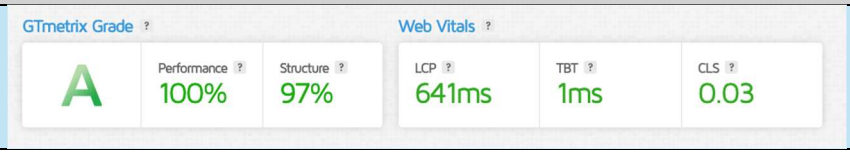
Original Game Name	Original Author / Publisher	Why it's usable?
Snake	Nokia	Nokia only holds copyright / patents on specifically their code, functionality, and art used for their game not the idea or general mechanics of snake.
Hangman	Undefined	There is no one owner of hangman, therefore if the code is originally produced there are no copyright concerns.

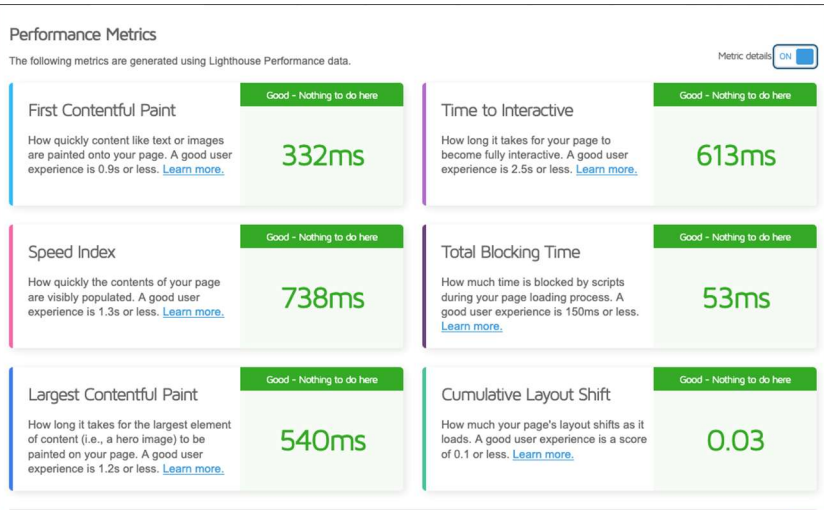
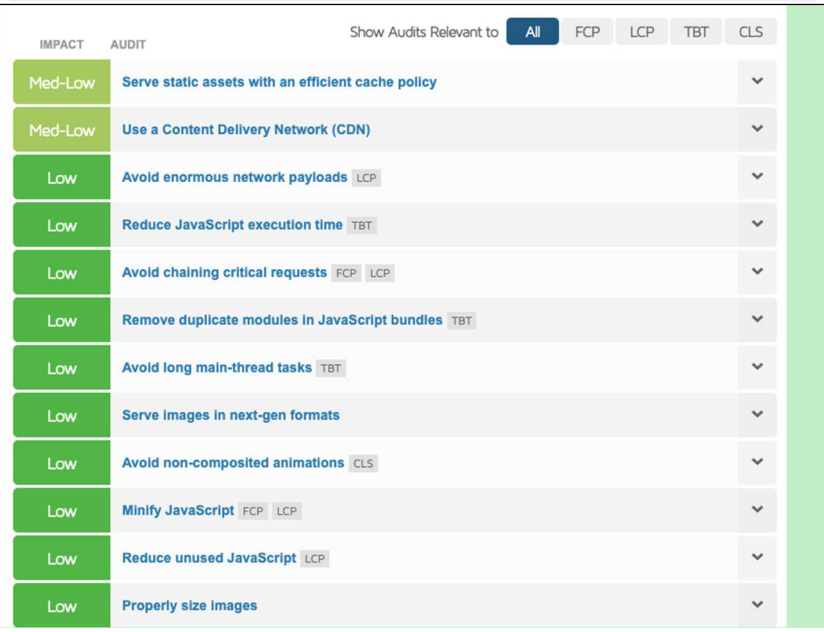
Our 3rd party dependencies are listed below –

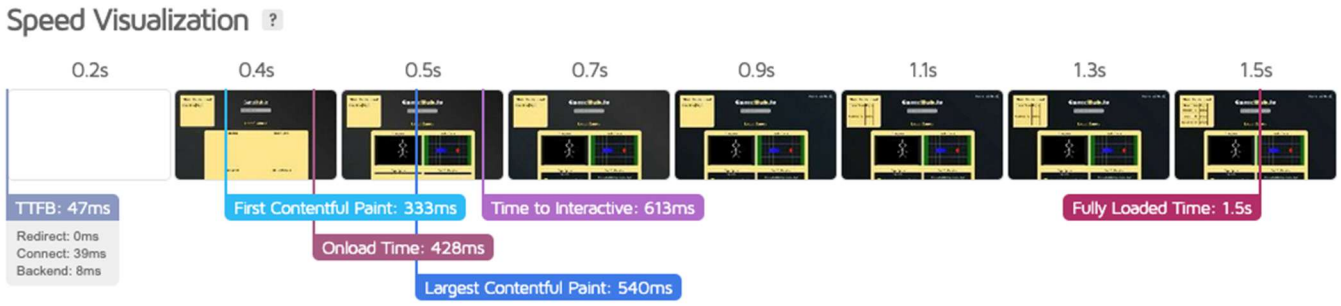
Dependency	Version	License
Firebase (JavaScript SDK)	v12.12.1	Apache License 2.0
SweetAlert2	v11.26.24	MIT License

Metrics –

Using an industry recognized tool *GTMetrix* we were able to collect standardized performance metrics for our website.

Test	Output	Test Screenshot
GTMetrix Overall	Grade: A	

Performance Metrics	Average of **45% faster** than the minimum acceptable speeds	
Structure Metrics	Grade: 97%	



Through our team's planning of our website structure, we have managed to ensure the loading order of assets is optimized to prevent any waiting times for previous assets. While our frame data is great there are some things to note as far as improvement. The styles and assets for our website could use some small restructuring to make sure that we are only loading necessary data for each webpage. This could be done by both refactoring our CSS file to separate files for each section of the website as well as redesigning the "Search UI" functionality to predict the static assets that will be needed for display before they are needed.

Code Summary –

Date: 2026-04-27 18:51:55

Directory: GameHub.io/*

Total: 29 files, 2465 codes, 407 comments, 541 blanks, all 3413 lines

Languages –

language	files	code	comment	blank	total
JavaScript	12	1,349	202	291	1,842
HTML	12	566	175	156	897
PostCSS	1	475	30	77	582
JSON	3	73	0	16	89
Markdown	1	2	0	1	3

Directories –

path	files	code	comment	blank	total
.	29	2,465	407	541	3,413
. (Files)	3	29	0	3	32
web_page	26	2,436	407	538	3,381
web_page (Files)	3	832	95	140	1,067
web_page/games	13	1,118	147	220	1,485
web_page/games/hangman	3	307	34	44	385
web_page/games/path_to_paradise	3	159	20	39	218
web_page/games/snake	4	369	59	107	535
web_page/games/test_game	1	21	6	2	29
web_page/games/type_sprint	2	262	28	28	318
web_page/htmls	6	309	119	115	543
web_page/scripts	4	177	46	63	286

System Requirements –

The following minimum system specifications are required to access and operate the GameHub.io application. These requirements apply to end users accessing the system via a web browser.

Operating System: Windows 10 or later, macOS 11 (Big Sur) or later, or a modern Linux distribution.

Web Browser: Google Chrome (v100+), Mozilla Firefox (v100+), Microsoft Edge (v100+), or Safari (v15+). Internet Explorer is not supported.

Internet Connection: A stable broadband connection with a minimum speed of 5 Mbps is recommended for optimal performance.

Display: A resolution of 1920×1080 or greater is recommended.

Browser Settings: JavaScript and cookies must be enabled. Pop-up blockers should be configured to allow pop-ups from "cs3300-groupproject.web.app".

Mobile Devices: The application is compatible with iOS 15 or later and Android 10 or later, accessed via the device's default or supported browser.

Reviewing Requirements –

As a team we managed to build, test, and implement **22/22** of our *minimum viable product* requirements. These requirements include but are not limited to, outlining, planning, and programming the primary website pages and navigation throughout, designing, planning and programming both visual and logical functionality for 4 games (Hangman, Snake, TypeSprint, and Path to Paradise), designing and programming logical ties between score tracking for each applicable game and the live leaderboards visible to all users and creating a dynamic, searchable, ad-free space for users to play and compete in these games while saving their data by signing in to our platform. While our team was successful in delivering our *MVP,* we only managed to deliver **28/33** total requirements we outlined for this project.

2.1.2	Implement light/dark themes for games	whole team	Brandon Sith	Ver 2.1	18-Apr	N/A
2.2.1	Design/Build games 4-6	whole team	Brandon/David	Ver 2.2	22-Apr	N/A
2.2.2	Implement canvas display for games 4-6	whole team	Brandon/Trey	Ver 2.2	22-Apr	N/A
2.2.4	Implement password reset	whole team	Trey/Azam	Ver 2.2	22-Apr	N/A
3.0.1	Implement server run multiplayer game	whole team	Whole Team	Ver 3.0	22-Apr	N/A

Diving into the requirements that we set for ourselves we can see clearly that there were no requirements that fell under *minimum viable product* that were not completed. We have 5 requirements that did not get implemented including designing/programming/implementing games 5 & 6 and their canvas', password resetting, light/dark theme toggles for website & games, and programming/implementing a server run multiplayer game. As these requirements were not necessary for our *MVP* it is acceptable for these to be unimplemented at this time. We can consider all necessary requirements to be implemented and fulfilled.

Software Verification -

Our team planned for 18 unit test, 12 integration tests and 3 system tests, totaling 33 tests for 33 requirements. Out of these tests we completed 16/18 unit tests, 10/12 integration tests, and 2/3 system tests because of uncompleted requirements.

Test Type	Number Tests Planned	Number of Tests Executed
Unit Test	18	16
Integration Test	12	10
System Test	3	2

This lets us deduce that our team had a requirement implementation and testing ratio of 28/32 or 84% of total requirements.

Current Code Condition -

All the things we flagged during development have been fixed. That being said, there are a few things we would have liked to implement/polish that we were unable to do. For our requirements, our goal was to have 4-6 games on the website. We completed this to the bare minimum with 4 games. We would have liked to have more, but time was a limitation for us. Similar to this last requirement, we wanted to have canvas displays for 4-6 games. Since we only have 4 games, this requirement is also incomplete. Another requirement we were unable to fulfill was the password reset feature. This feature ended up getting expanded a little bit. Our hopes were to be able to edit account info such as username, password, and display name. Unfortunately, we didn't have time to get this one done either. Some other goals we fell short of were having a like/dark theme for the games and website. Currently we only have a dark theme because the natural color scheme of the website is darker colors. The last requirement we didn't complete was a server run multiplayer game. This game was kind of a stretch for us in the first place, so there's no surprise that we didn't get to it. We originally had this as an optional requirement to complete only if we had time.

All of our games and website elements are pretty polished. We don't have any known bugs at the moment, and all the issues with the website stem from the incomplete requirements stated above. Even though we didn't complete 100% of our requirements, the website is still operational. These missing requirements do not hinder the base functionality of the website.

Team Membership and Roles:

Gardner, Trey – Front-end Developer & Backup Game Developer:

Designed and programmed all the front-end structure of the website including the homepage, sign-in, sign-up, leaderboard, and account settings pages. Designed and programmed the searchable UI for the entire website on the homepage. Designed the visual and logical layout for all leaderboards on the website providing the building blocks for the back-end implementations. Programmed and managed user page states to create a dynamic experience that contrasts between a guest and a signed in user. Programmed the UI and logic behind both "TypeSprint" and "Path to Paradise" games. Provided data structures for back-end implementation of score tracking and leaderboard update for "TypeSprint". Created and managed the entire file tree for the project. Managed version rollbacks, git branches, and branch merges throughout the project.

Jaffer, Azam – Back-end Developer & Database Engineer

Designed, programmed, and managed the back-end infrastructure for website deployment. Designed and implemented the database schema for securely storing user account data and game scores. Programmed the sign-in and sign-up logic to communicate with the Firebase back-end, ensuring secure authentication and signed-in state management for personalized data handling. Built the score storage and retrieval systems powering leaderboard display across the website.

Sith, Brandon – Pair Programmer & UI/UX

Worked with both David and Azam over review of final product and Test Planning. Worked on optimizations for hangman & snake, shortening of code for better readability. Handled system testing of general canvas and html test files. Pair Programmed with other game developers to finish requirements.

Zarkovacki, David – Game Developer:

Programmed the game logic, display, and html files for hangman and snake. After base games were completed, the team tested them so they could provide feedback. Fixed all known bugs and polished games to provide a better user experience.